

Chapter 2: Image Basics

Assoc. Prof. Dr.-Ing. Nguyễn Xuân Hà

Robotics Group, Dept. of Mechatronics, SME - HUST

ha.nguyenxuan@hust.edu.vn

<https://sites.google.com/site/tsnguyenxuanhabkhn/home>

Lecture 2: Programming exercise

Bài 1: Cho ảnh như hình dưới đây (ảnh 'C2_B1.png' trong thư mục data đi kèm).



- a) Viết đoạn chương trình sử dụng thư viện opencv để mở và hiển thị hình ảnh.
- b) Viết đoạn chương trình sử dụng thư viện matplotlib để hiển thị hình ảnh.
- c) Viết đoạn chương trình hiển thị kích thước của hình ảnh đọc được và hiển thị từng kênh màu của ảnh.
- d) Viết đoạn chương trình chuyển từ ảnh màu sang ảnh xám theo hai cách: sử dụng câu lệnh có sẵn trong opencv và công thức chuyển đổi. So sánh hai kết quả này

Lecture 2: Programming exercise

Các thư viện cần sử dụng

<code>import numpy as np</code>	→ Thư viện tính toán dữ liệu vector, ma trận
<code>import cv2</code>	→ Thư viện phục vụ xử lý ảnh và thị giác máy tính
<code>import matplotlib.pyplot as plt</code>	→ Thư viện phục vụ hiển thị, vẽ đồ thị

Lecture 2: Programming exercise

Đọc và hiển thị hình ảnh bằng thư viện opencv

image_dir = './data/C2_B1.png' → Đường dẫn đến file ảnh

img = cv2.imread(image_dir) → Lệnh đọc ảnh và lưu dữ liệu vào biến img

cv2.imshow('LOGO', img) → Hiển thị hình ảnh trong một cửa sổ riêng biệt
→ 'LOGO': tên hiển thị trên cửa sổ
→ img: biến lưu trữ hình ảnh

cv2.waitKey(0) → Sử dụng để duy trì cửa sổ hình ảnh cho đến khi đóng lại

cv2.destroyAllWindows() → Đóng tất cả các cửa sổ opencv

Lecture 2: Programming exercise

Đọc và hiển thị hình ảnh bằng thư viện opencv



Lecture 2: Programming exercise

Hiển thị hình ảnh bằng thư viện matplotlib

`plt.figure()` → Tạo cửa sổ hiển thị mới

`plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))` → Hiển thị hình ảnh

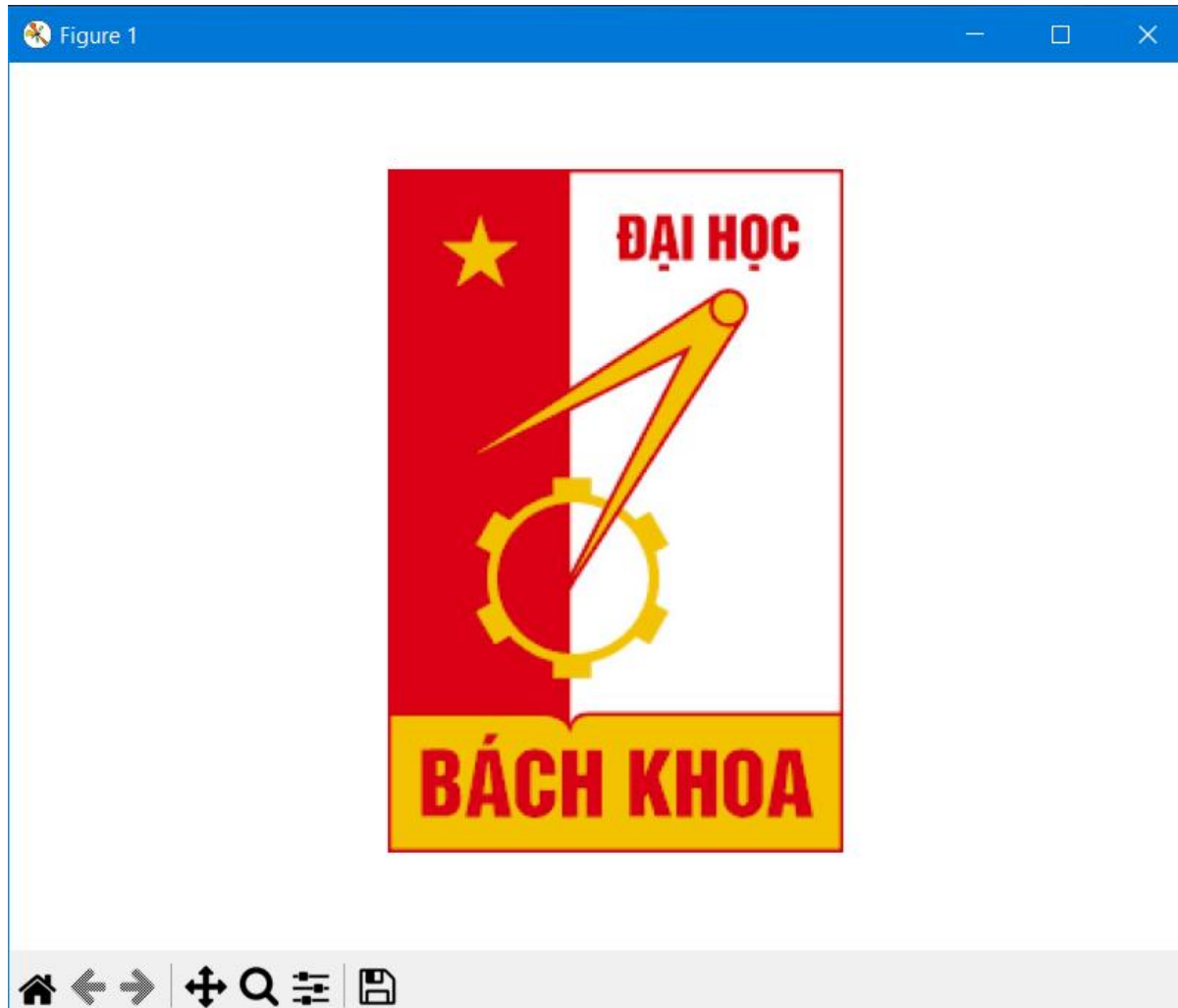
`plt.axis('off')` → Chuyển màu từ thứ tự BGR về RGB

`plt.show()` → Tắt hiển thị đơn vị trục

`plt.show()` → Hiển thị cửa sổ vừa tạo

Lecture 2: Programming exercise

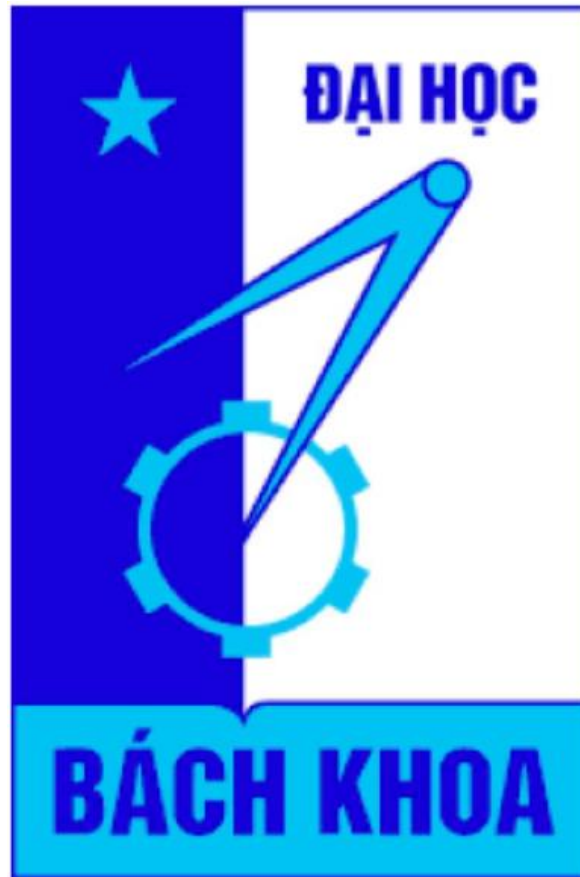
Hiển thị hình ảnh bằng thư viện matplotlib



Lecture 2: Programming exercise

Hiển thị hình ảnh bằng thư viện matplotlib

Nếu không chuyển đổi kênh màu thì kết quả như nào?



Lecture 2: Programming exercise

Dữ liệu của ảnh đọc từ opencv

Dữ liệu ảnh màu khi được đọc bởi câu lệnh

```
img = cv2.imread(image_dir)
```

Biến `img` có dạng là một vector ba chiều, với các kích thước như sau (sử dụng `img.shape`):

```
(275, 183, 3)
```

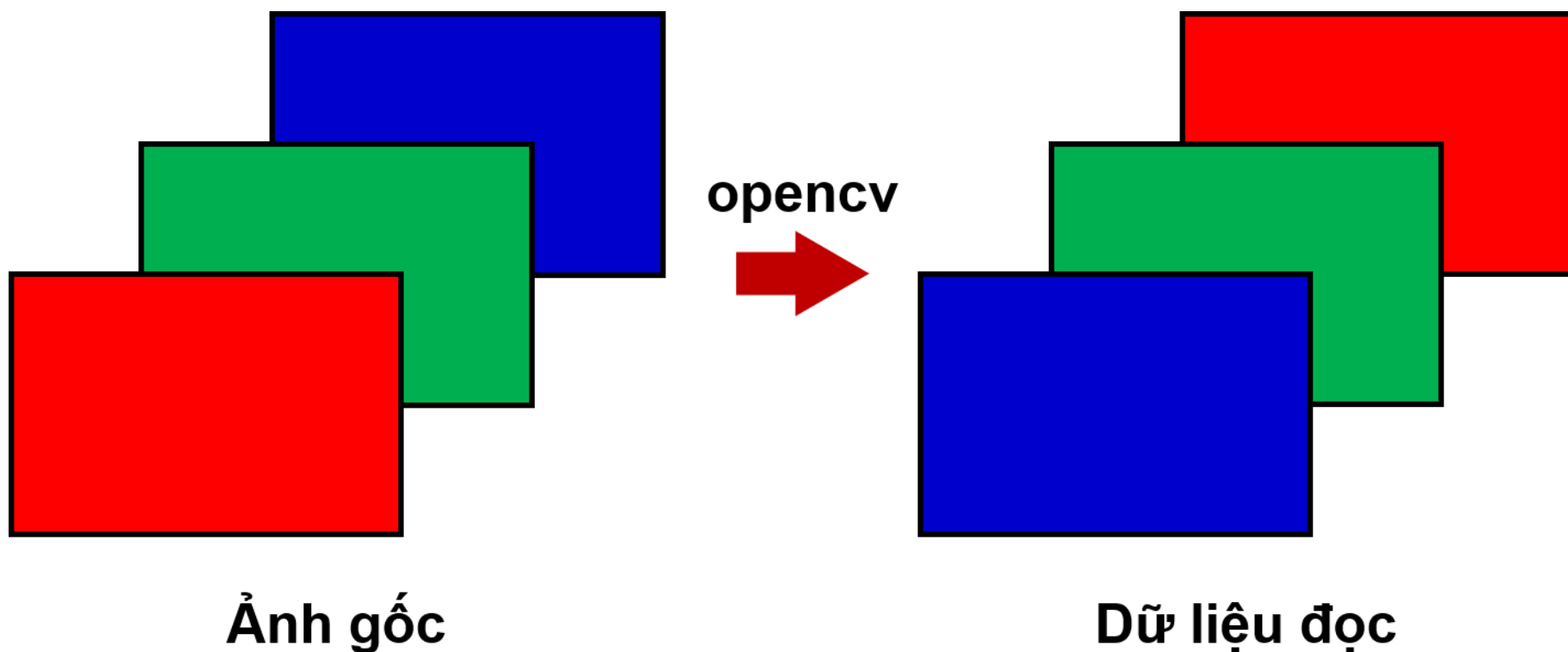
Trong đó:

- 275 – số hàng (chiều cao H)
- 183 – số cột (chiều rộng W)
- 3 – số kênh màu

Lecture 2: Programming exercise

Dữ liệu của ảnh đọc từ opencv

Chú ý: khi sử dụng câu lệnh `cv2.imread()`, các kênh màu của ảnh ban đầu là **R – G – B** được chuyển sang **B – G – R**.

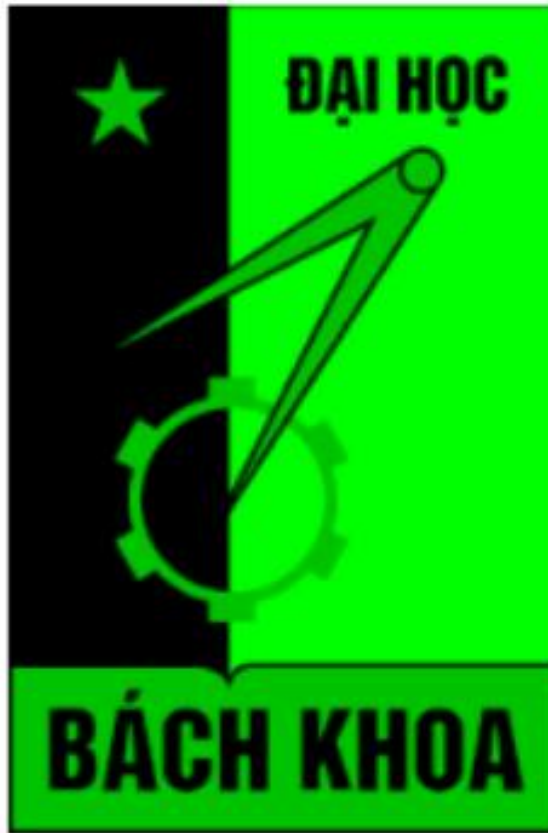


Lecture 2: Programming exercise

Hiển thị các kênh màu của ảnh.



```
img_red = img.copy()  
img_red[:, :, 0] = 0  
img_red[:, :, 1] = 0
```



```
img_green = img.copy()  
img_green[:, :, 0] = 0  
img_green[:, :, 2] = 0
```



```
img_blue = img.copy()  
img_blue[:, :, 1] = 0  
img_blue[:, :, 2] = 0
```

Lecture 2: Programming exercise

Chuyển ảnh màu sang ảnh xám

```
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Chuyển ảnh màu sang ảnh xám

```
img_gray1 = np.zeros((img.shape[0], img.shape[1]))
```

Tạo mảng lưu trữ cho tính toán

```
img_gray1[:, :] = 0.299*img[:, :, 2] +  
0.587*img[:, :, 1] + 0.114*img[:, :, 0]
```

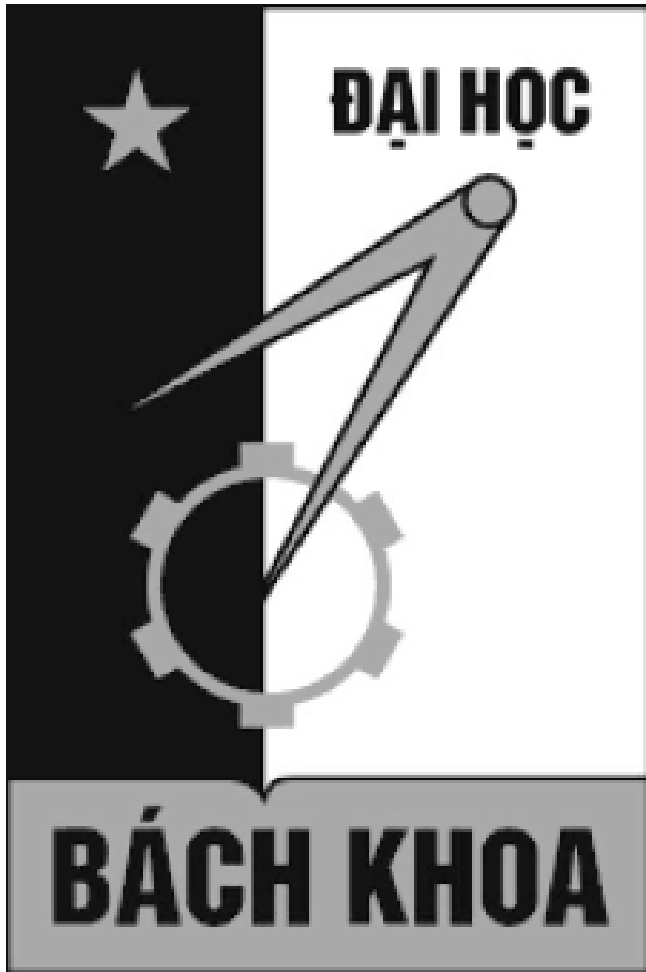
Công thức chuyển đổi

```
img_gray1 = img_gray1.astype(np.uint8)
```

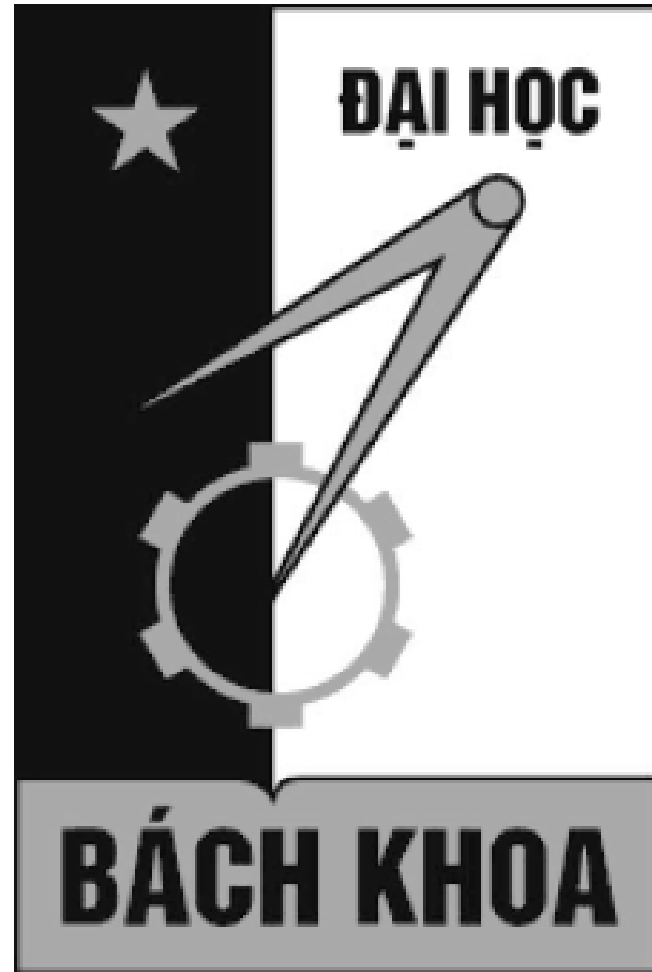
Đưa giá trị thực về giá trị nguyên

Lecture 2: Programming exercise

Chuyển ảnh màu sang ảnh xám



Câu lệnh `cv2.cvtColor()`



Công thức

Lecture 2: Programming exercise

Bài 2: Cho ảnh như hình dưới đây (ảnh 'C2_B2.png' trong thư mục data đi kèm).



Viết chương trình sử dụng phương pháp nội suy điểm gần nhất (nearest neighbor interpolation) và nội suy song tuyến (bilinear interpolation) để phóng to ảnh ra với kích thước mỗi chiều được nhân lên 2 lần. So sánh kết quả với câu lệnh có sẵn trong opencv

Lecture 2: Programming exercise

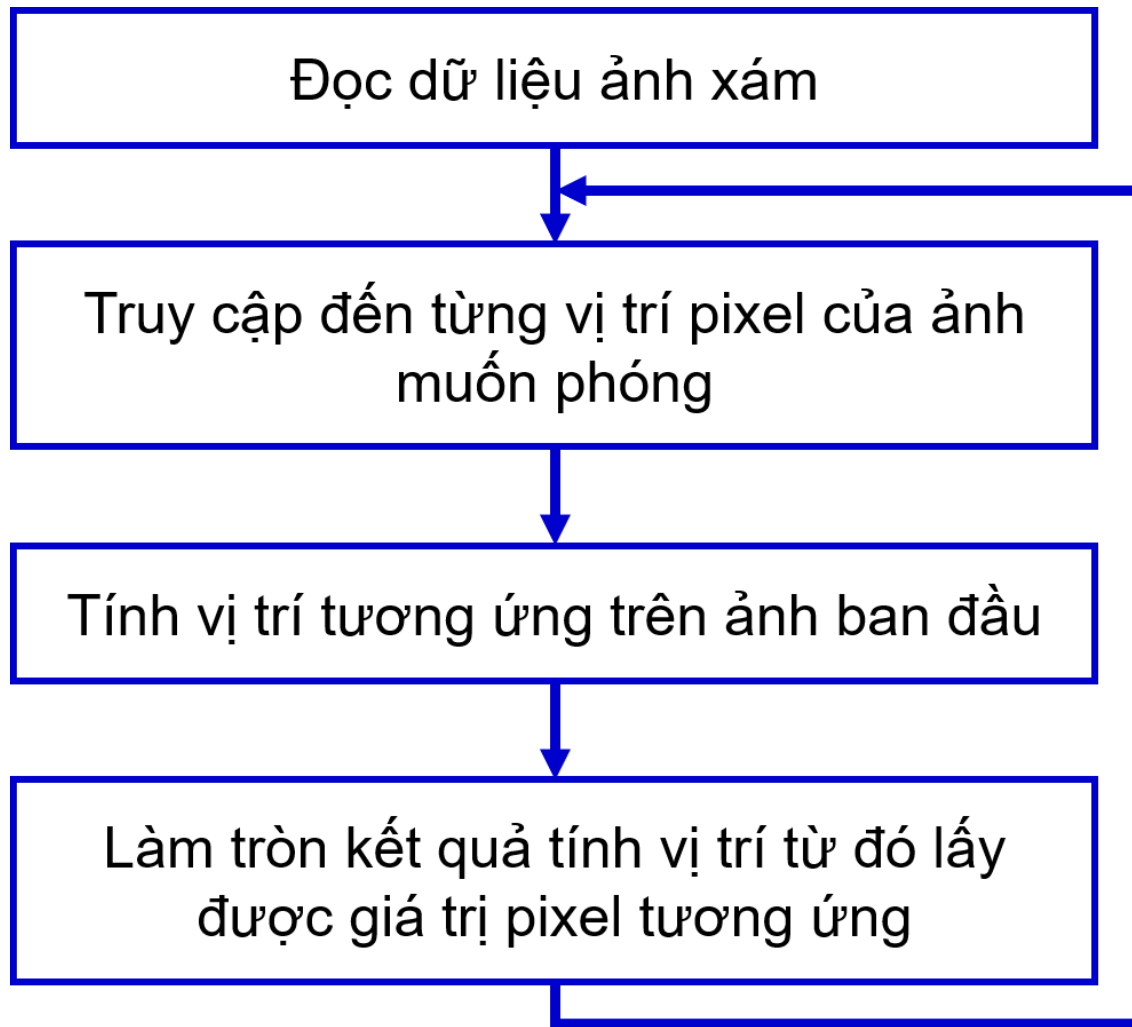
Ý tưởng sử dụng phép tính toán ngược: từ vị trí pixel trong ảnh cần tìm truy ngược lại vị trí pixel trên ảnh gốc.

Công thức liên hệ giữa tọa độ hai bên:

$$\begin{cases} \frac{x_1}{W_1 - 1} = \frac{x_2}{W_2 - 1} \\ \frac{y_1}{H_1 - 1} = \frac{y_2}{H_2 - 1} \end{cases} \rightarrow \begin{cases} x_1 = \frac{W_1 - 1}{W_2 - 1} \times x_2 \\ y_1 = \frac{H_1 - 1}{H_2 - 1} \times y_2 \end{cases}$$

Lecture 2: Programming exercise

Thuật toán phóng to ảnh sử dụng nội suy gần nhất



Lecture 2: Programming exercise

Chương trình

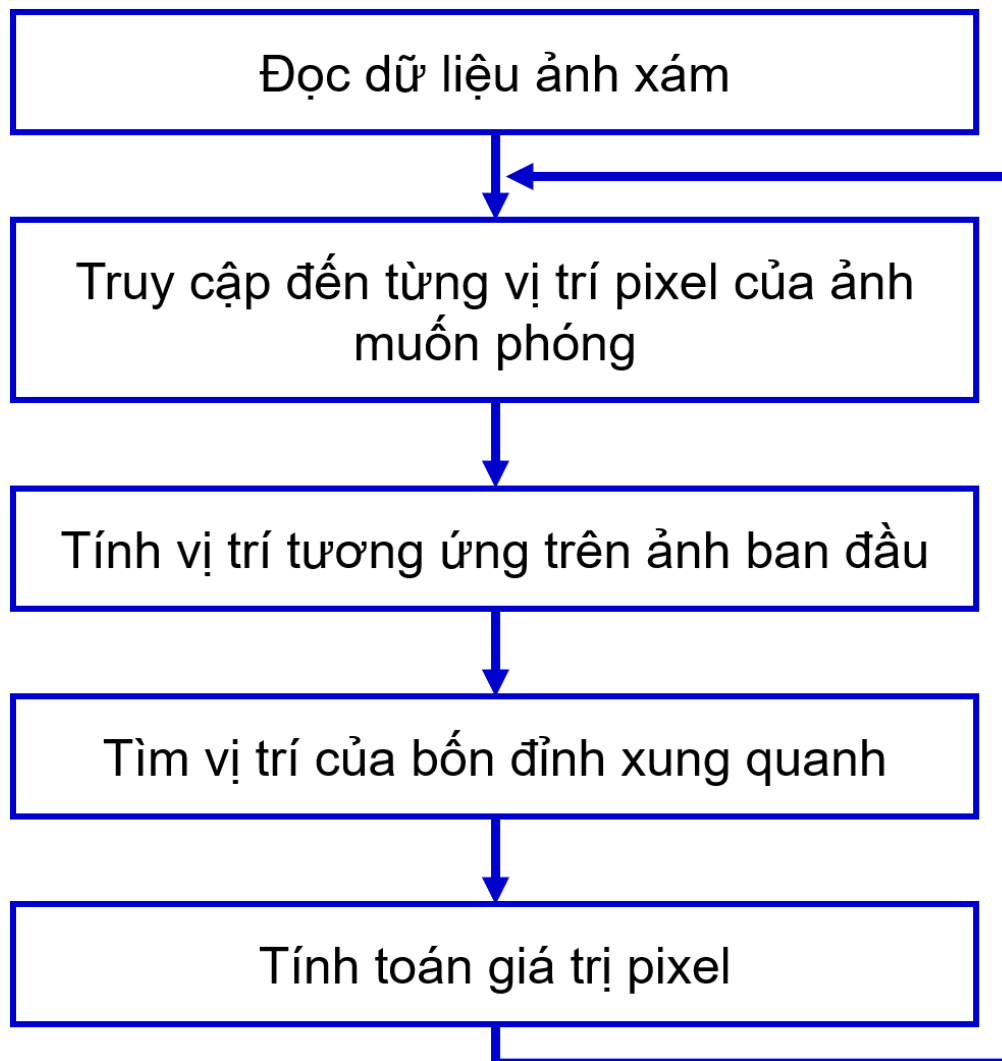
```
img2 = np.zeros((H2, W2)) -----> Tạo biến chứa ảnh phóng to
for i in range(H2):
    for j in range(W2): -----> Truy cập đến từng phần tử

        y = int (i/(H2-1)*(H1-1)) -----> Tính các vị trí tương ứng
        x = int (j/(W2-1)*(W1-1)) -----> với ảnh ban đầu

    img2[i, j] = img1[y, x] -----> Lấy giá trị pixel từ ảnh gốc
```

Lecture 2: Programming exercise

Thuật toán phóng to ảnh sử dụng nội suy song tuyến



Lecture 2: Programming exercise

Chương trình

```
for i in range(H2):  
    for j in range(W2):  
        y = i/(H2-1)*(H1-1)  
        x = j/(W2-1)*(W1-1)  
        if x == W1-1:  
            x1 = m.floor(x-1)  
            x2 = x1 + 1  
        else:  
            x1 = m.floor(x)  
            x2 = x1 + 1  
        if y == H1-1:  
            y1 = m.floor(y-1)  
            y2 = y1 + 1  
        else:  
            y1 = m.floor(y)  
            y2 = y1 + 1
```

Truy cập đến từng phần tử

Tính các vị trí tương ứng với ảnh ban đầu

Tìm vị trí của bốn đỉnh xung quanh

Lecture 2: Programming exercise

Chương trình

```
for i in range(H2):  
    for j in range(W2):  
        ...  
        Q11 = img1[y1, x1]  
        Q12 = img1[y2, x1]  
        Q21 = img1[y1, x2]  
        Q22 = img1[y2, x2]  
        m1 = np.array([x2 - x, x - x1])  
        m2 = np.array([[Q11, Q12], [Q21, Q22]])  
        m3 = np.array([y2 - y, y - y1])  
        Q = np.dot(np.dot(m1, m2), m3)  
        img2[i, j] = Q[0]
```

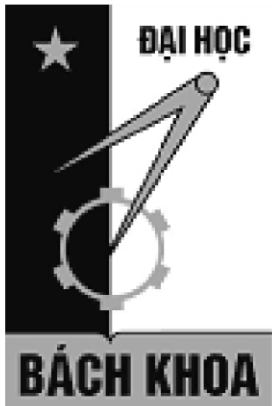
Tính toán giá trị pixel

`img2 = img2.astype(np.uint8)` -----> Đưa giá trị thực về giá trị nguyên

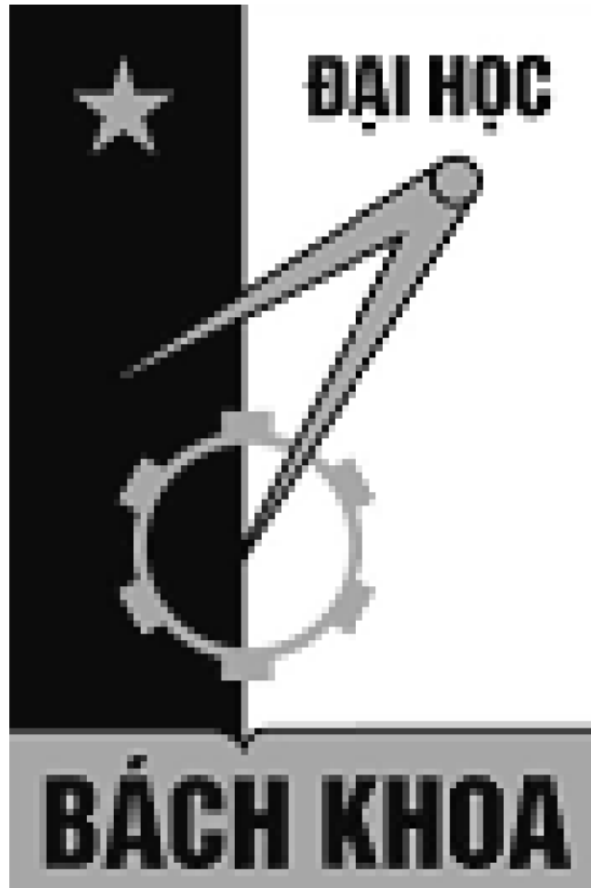
Lecture 2: Programming exercise

Kết quả

Origin



Nearest Neighbor



Bilinear



Lecture 2: Programming exercise

Câu lệnh thực hiện trong opencv

```
img_nn1 = cv2.resize(img_gray, None,
                    fx=2, fy=2,
                    interpolation = cv2.INTER_NEAREST)

img_bi1 = cv2.resize(img_gray, None,
                    fx=2, fy=2,
                    interpolation = cv2.INTER_LINEAR)
```

Kích thước ảnh mới

Tỷ lệ phóng to ở các chiều

Thuật toán nội suy gần nhất

Thuật toán nội suy tuyến tính

Lecture 2: Programming exercise

Bài 3: Cho ảnh như hình dưới đây (ảnh 'C2_B3.png' trong thư mục data đi kèm).



Viết chương trình chuyển đổi ảnh trên sang ảnh xám và vẽ biểu đồ biên độ tần số ảnh histogram của ảnh xám thu được

Lecture 2: Programming exercise

Chương trình

```
his = np.zeros(256)
for i in range(img_gray.shape[0]):
    for j in range(img_gray.shape[1]):
        val = img_gray[i, j]
        his[val] += 1
```

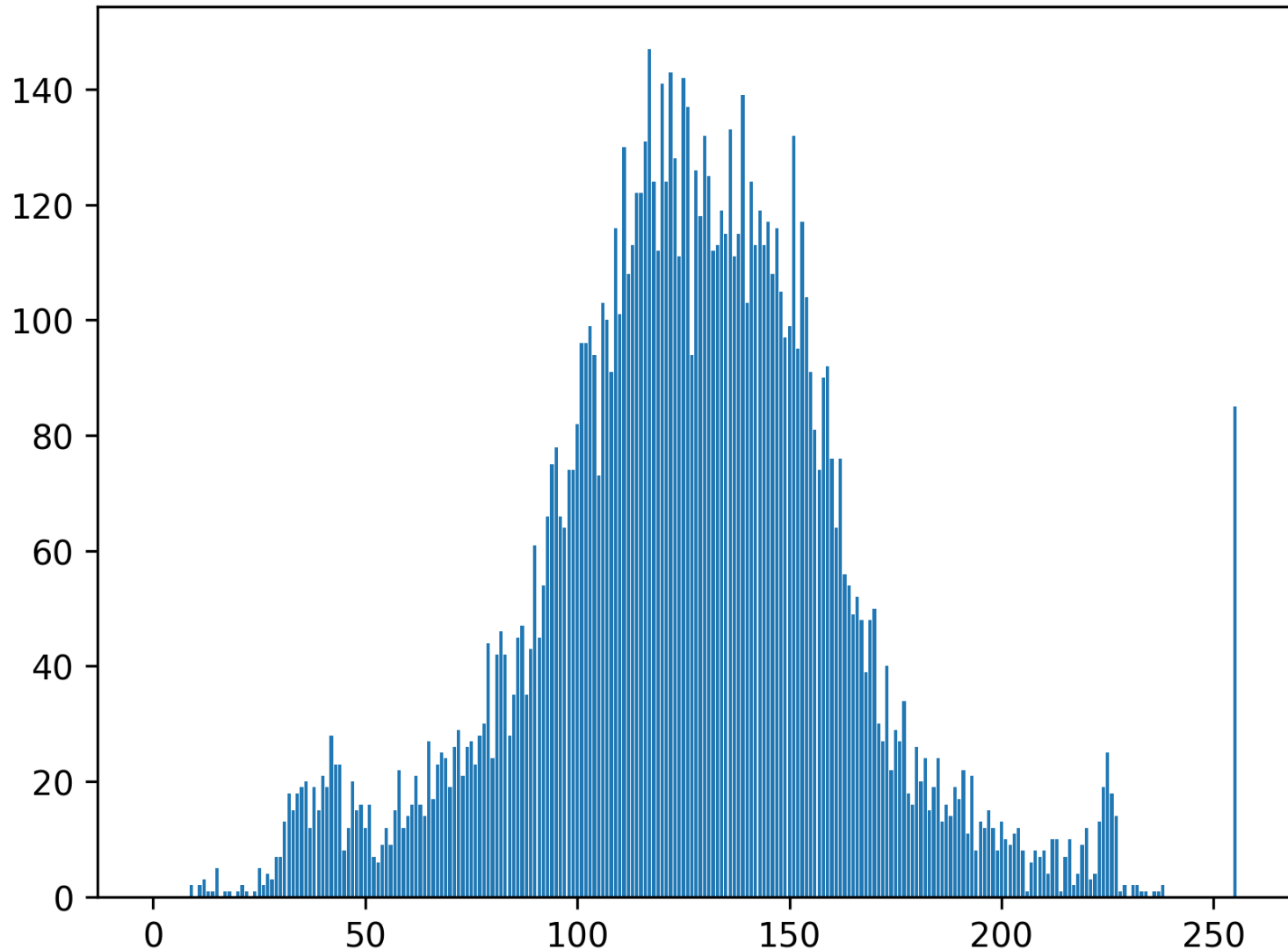
Khởi tạo mảng chứa giá trị ban đầu

Duyệt từng vị trí pixel của ảnh

Vị trí tương ứng với giá trị pixel tăng thêm một đơn vị

Lecture 2: Programming exercise

Kết quả



Lecture 2: Programming exercise

Bài 4: Cho ảnh như hình dưới đây (ảnh 'C2_B4.png' trong thư mục data đi kèm).



Viết chương trình sử dụng phép quay để quay ảnh đi một góc là 30° . Hiện thị ảnh kết quả thu được

Lecture 2: Programming exercise

Công thức của phép quay từ điểm có tọa độ (v, w) thành điểm có tọa độ (x, y) :

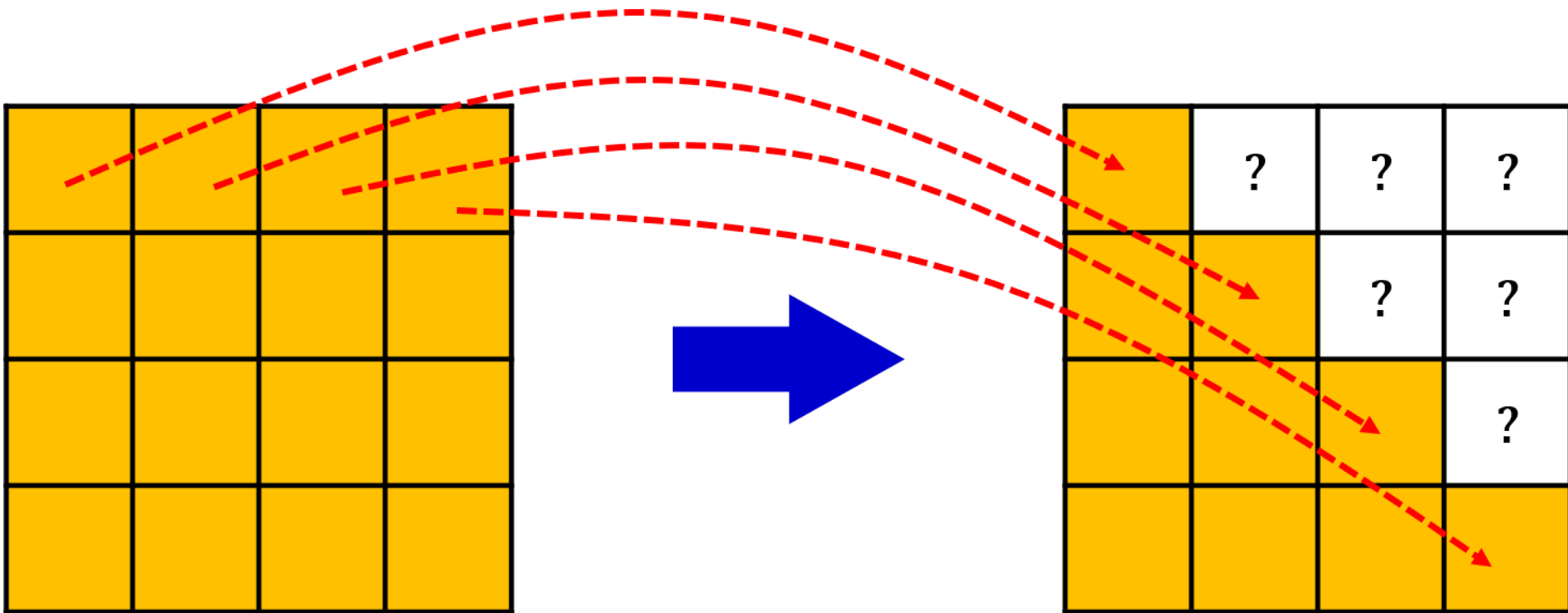
$$\begin{bmatrix} x & y & 1 \end{bmatrix} = \begin{bmatrix} v & w & 1 \end{bmatrix} \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Trong đó:

- x, v là vị trí cột;
- y, w là vị trí hàng.

Lecture 2: Programming exercise

Áp dụng công thức quay



Vẫn còn các vị trí còn trống, cần xử lý như nào?

Lecture 2: Programming exercise

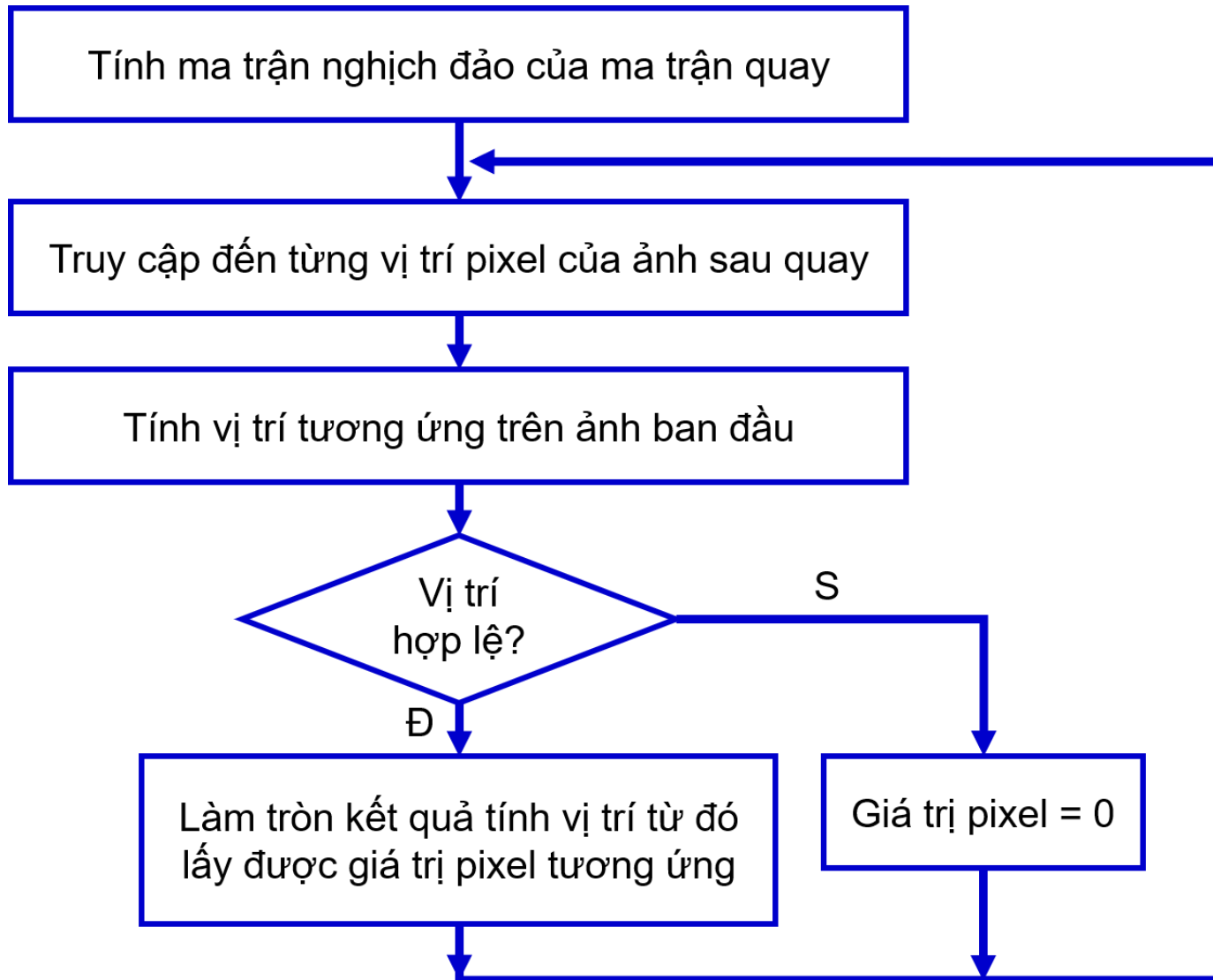
Để có thể tính giá trị pixel tại tất cả các vị trí của ảnh mới, ý tưởng được sử dụng là truy ngược lại vị trí ban đầu tương tự như phần phóng to ảnh.

$$[x \quad y \quad 1] = [v \quad w \quad 1] \times R$$

$$\Rightarrow [v \quad w \quad 1] = [x \quad y \quad 1] \times R^{-1}$$

Lecture 2: Programming exercise

Thuật toán quay ảnh sử dụng nội suy gần nhất



Lecture 2: Programming exercise

Chương trình

```
RotMatrix = np.array([
    [np.cos(angle), np.sin(angle), 0],
    [-np.sin(angle), np.cos(angle), 0],
    [0, 0, 1]
])
RotInv = np.linalg.inv(RotMatrix)

for w in range(H1):
    for v in range(W1):

        new_vec = np.array([v, w, 1])
        old_vec = np.dot(new_vec, RotInv)

        x, y = old_vec[0], old_vec[1]
        if 0 <= x and x < W1 and 0 <= y and y < H1:
            img2[w, v] = img1[m.floor(y), m.floor(x)]
```

Ma trận quay

Ma trận nghịch đảo

Duyệt từng vị trí pixel của ảnh

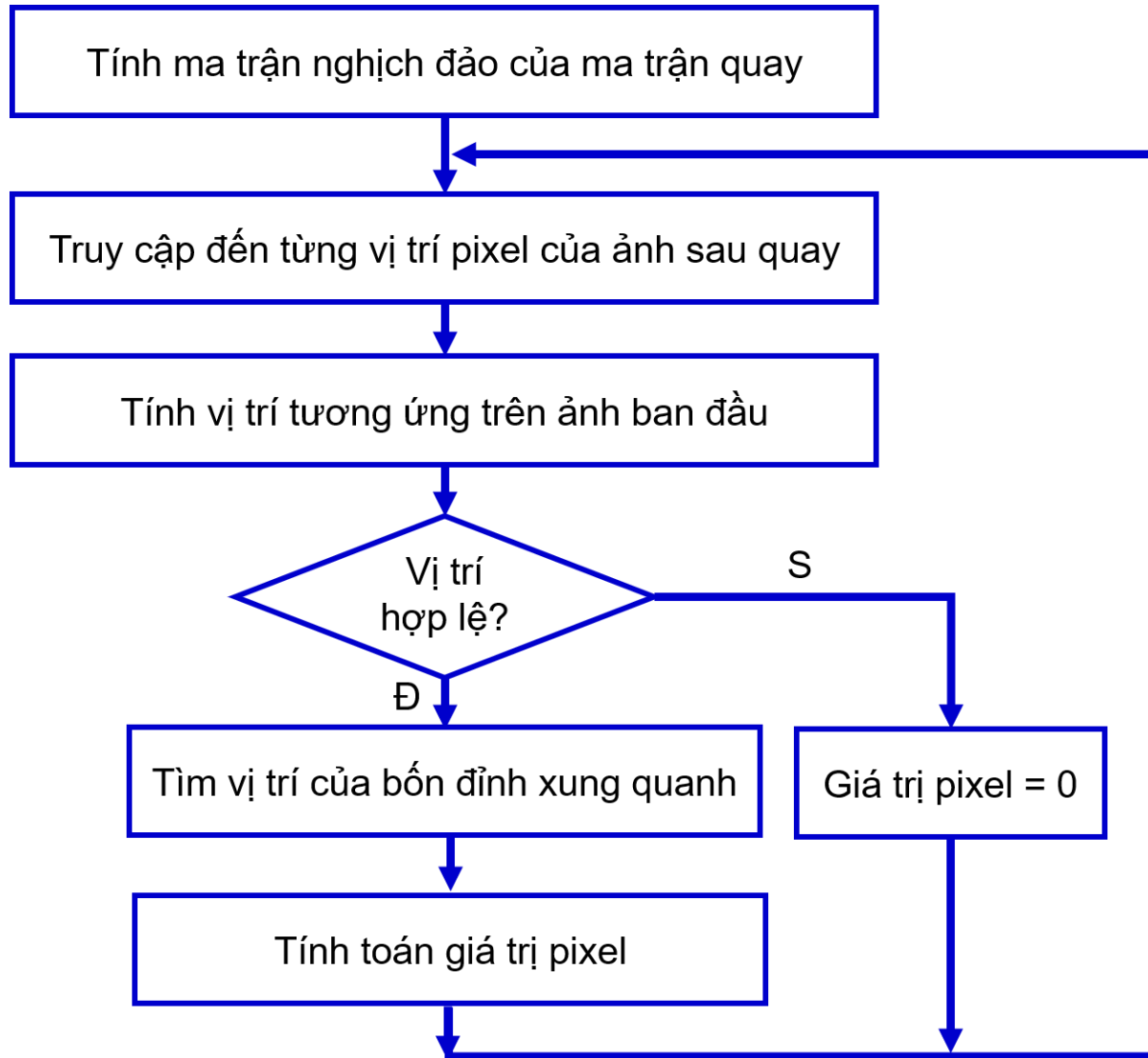
Vector tọa độ pixel mới

Vector tọa độ pixel ban đầu

Kiểm tra tọa độ hợp lệ và tính toán giá trị pixel

Lecture 2: Programming exercise

Thuật toán quay ảnh sử dụng nội suy song tuyến



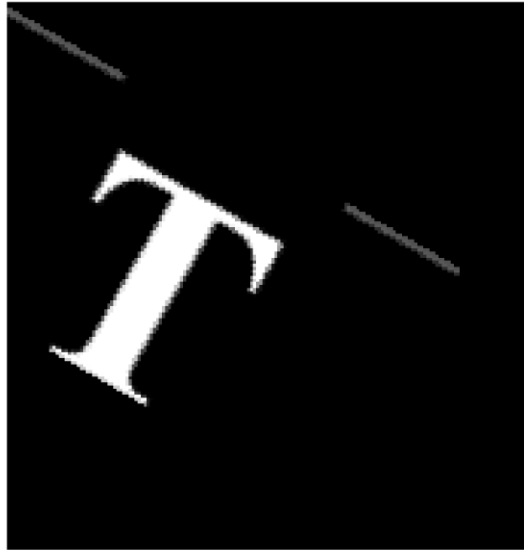
Lecture 2: Programming exercise

Kết quả

Origin



Nearest Neighbor



Bilinear



Lecture 2: Programming exercise

Câu lệnh của opencv

```
rows, cols = img_gray.shape
```

```
M = cv2.getRotationMatrix2D((0, 0), -30, 1)
```

Tạo ma trận quay

Tỷ lệ so với ảnh cũ

Góc quay

Vị trí quay

```
dst = cv2.warpAffine(img_gray, M, (cols,rows))
```

Tính toán ảnh mới

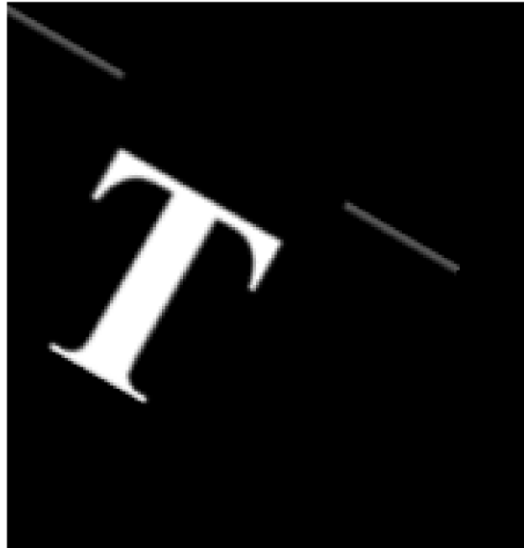
Lecture 2: Programming exercise

Kết quả

Origin



Bilinear



opencv



Lecture 2: Programming exercise

Bài 5: Cho ảnh như hình dưới đây (ảnh 'C2_B5.png' trong thư mục data đi kèm).



Viết chương trình tính toán kết quả đầu ra khi đưa ảnh trên qua các bộ lọc trung bình, bộ lọc gaussian ($\sigma=10$), bộ lọc trung vị. Các bộ lọc này đều có kích thước là 7×7 . So sánh các kết quả này với nhau.

Lecture 2: Programming exercise

Chương trình

```
K_mean = np.ones((7, 7))  
K_mean = K_mean / 49
```

Ma trận lọc

```
I1 = cv2.filter2D(img_gray,
```

Câu lệnh lọc ảnh sử dụng phép tích chập

-1,

Chiều sâu ảnh đầu ra

Giá trị '-1' $\approx$ giống ảnh đầu vào

K_mean,

Ma trận lọc ảnh

Cách xử lý các đường biên

Giá trị này ở đây là thêm viền là các giá trị 0

```
borderType = cv2.BORDER_CONSTANT)
```

Lecture 2: Programming exercise

Chương trình

```
I2 = cv2.GaussianBlur(img_gray,
```

`(7, 7),` → Kích thước ma trận lọc

`10,` → Giá trị σ

Cách xử lý các đường biên
Giá trị này ở đây là thêm viền là các giá trị 0

`borderType = cv2.BORDER_CONSTANT)` ↑

```
I3 = cv2.medianBlur(img_gray, 7) → Kích thước ma trận lọc
```

Lecture 2: Programming exercise

Kết quả

Origin



Average



Gaussian



Median

